Digital Energy Solutions
Power Transmission & Distribution

SOFTWARE INSTALLATION ASSISTANT USING NVIDIA AGENTIQ

INTERNSHIP REPORT

Janani S
Harishmaa M
Jayashree K.A

CONTENTS

1. Introduction

1.1. Acknowledgement

We would like to express our heartfelt gratitude to **Mr. Biju P. K.** and **Ms. Kavitha M.** for their constant guidance, encouragement, and unwavering support throughout the course of our internship. Their valuable insights and timely feedback played a crucial role in shaping our ideas and transforming our prototype into a successful outcome.

We extend our sincere thanks to **Mr. Karnena Madana Gopal**, our project mentor, for patiently addressing our queries and providing continuous support at every stage of the project.

We are also grateful to the **IT team**, for their technical guidance and cooperation despite their demanding schedules.

Lastly, we would like to thank **L&T Construction Management** for providing us with the opportunity and a conducive environment to carry out this internship, and for the valuable exposure to industry practices.

1.2. Team Details

TEAM CODE:

STUDENT NAMES:

1. Janani S
2. Harishmaa M
3. Jayashree K A

Single Point of Contact (SPOC): KARNENA MADANA GOPAL

1.3. Abstract

This internship was undertaken at L&T Construction with the objective of developing a prototype system for automating software installation approvals within a corporate IT environment. The system aimed to streamline the process of handling employee requests, parsing approval emails, verifying pre-approved software, and triggering installations based on defined rules.

Our work involved designing and implementing a client-server architecture using FastAPI for the backend and HTML, CSS, Javascript for the admin interface. We also integrated an AI-based agent (NeMo Agent Toolkit / DeepSeek) to intelligently parse and extract information from formal email communications. The system ensured approvals were validated by matching parsed content with a list of pre-approved softwares and expected installation commands.

Additional components included input hash verification, device ID mapping, job file generation, and a polling-based client-side installer. Logs and reports were maintained to ensure traceability and transparency.

This project gave us hands-on experience in full-stack development, API design, document parsing using AI, and practical exposure to corporate IT practices. The prototype lays a foundation for future automation in enterprise software request workflows.

1.4. Objective

The objective of this internship project was to design and implement an intelligent software installation approval system using a client-server model. The goal was to automate the validation of approval emails, ensure secure software deployment, and reduce manual intervention in enterprise IT workflows.

1.5. Technology Stack

COMPONENT	LANGUAGE/FRAMEWORK	REMARKS
Admin Input Parser	Python + AgentIQ	Best for NLP/YAML/OpenAI
API Server	Python (FastAPI)	Lightweight, YAML integration is great
Job Generator	Python	Easiest YAML + JSON handling
Client Agent	C# /.NET	Good for Windows services and supports UNC paths

Installer Source	HTTPS via FastAPI	Works with or without LAN
Web Admin Dashboard	HTML, CSS, Javascript	Easy and quick implementation
Hash Verification	Python (server), C# (client)	Reliable + secure SHA256 check
Logging	C# (client)	Easy POST logging to the admin

1.6. Scope

- The Admin (server) side handles email parsing using AI agents (AgentIQ / DeepSeek) to extract relevant information from approval emails.
- Approval data is cross-verified against a pre-defined list of authorized software, expected hashes, and email-device mappings.
- On successful validation, job files are generated containing installation instructions and monitored by the client system.
- The Client (employee) side polls for job files, verifies installer hashes, and executes installation commands, logging all actions for audit.
- A web application is implemented for admin users to monitor and manage the process.

1.7. Delivery Plan

Start Date: 09.06.2025

End Date: 25.07.2025

MILESTONE	DELIVERABLES	TARGET DATE
UI WIREFRAME FINALIZATION	Prepare a clear and interactive UI wireframe based on final implementation scope.	01.07.2025
ARCHITECTURE FINALIZATION	Draft a network architecture diagram of the prototype.	25.06.2025
TECHNOLOGY STACK FINALIZATION	Finalize the technology stack to be used for the project.	19.06.2025
PROTOTYPE DEVELOPMENT	Development of a working prototype with frontend.	24.06.2025
TESTING OF PROTOTYPE	Testing the prototype based on predefined test cases and documenting the results.	24.06.2025

1.8. Reporting

REVIEW MEETING	DISCUSSION POINTS	DATE
Review Meeting 1	Architecture and approach were presented. Upon discussion, a new approach was proposed by the attendees.	19.06.2025
Review Meeting 2	The new approach was drafted into an architecture and presented to the team. An extensive report on compatible open source tools for the project was also presented.	21.06.2025
Review Meeting 3	Test cases were reviewed and techstack was finalized.	25.06.2025
Review Meeting 4	The wireframe was reviewed and finalized by the team.	01.07.2025
Final review	PPT presentation and Project live demo	07.08.2025

1.9. Closure Criteria

1. Submission of source codes and API keys.
2. Submission of all necessary documents.
3. Final demo to the team.
4. Approval from SPOC.

2. Website

This project is integrated with an interactive web application which features a mail upload option, update section, ticket information page, and installation logs page. While the interface remains mostly the same for both Deepseek and NVIDIA AgentIQ methods, the latter includes the Agent's raw output in the dashboard for better explainability.

2.1. Dashboard

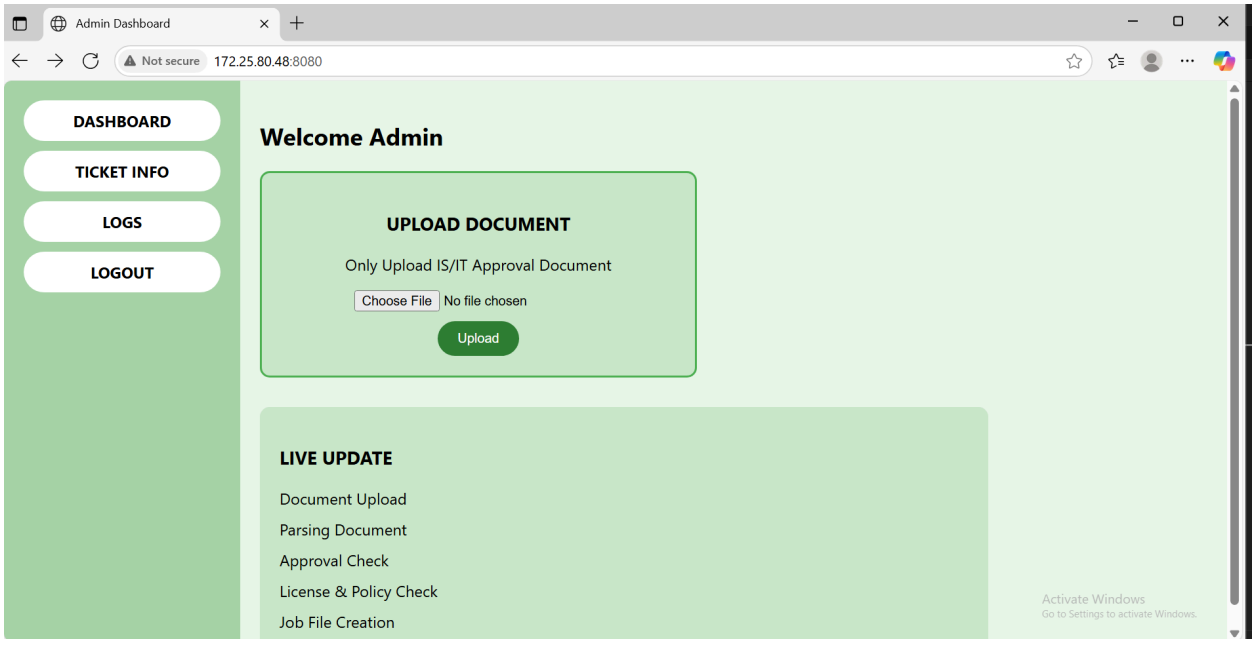


Fig. 1 Deepseek Dashboard

The dashboard consists of a mail upload option which supports .txt files, an update section and a sidebar menu.

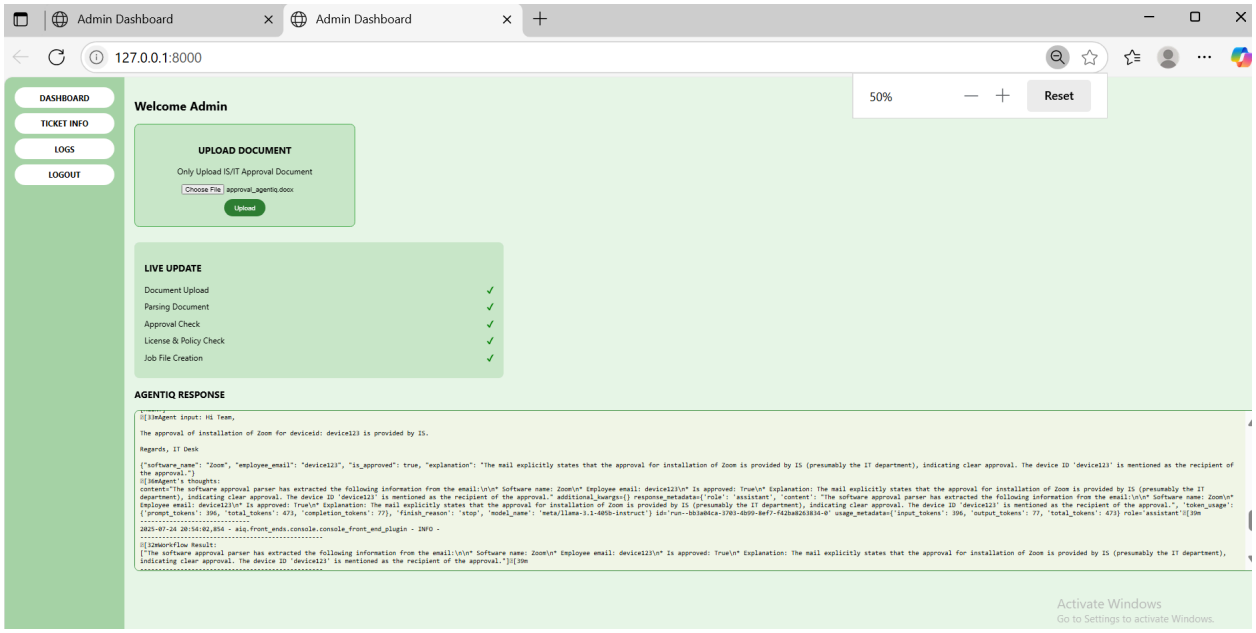


Fig. 2 AgentIQ dashboard

The AgentIQ dashboard features an extra section displaying the Agent's raw output.

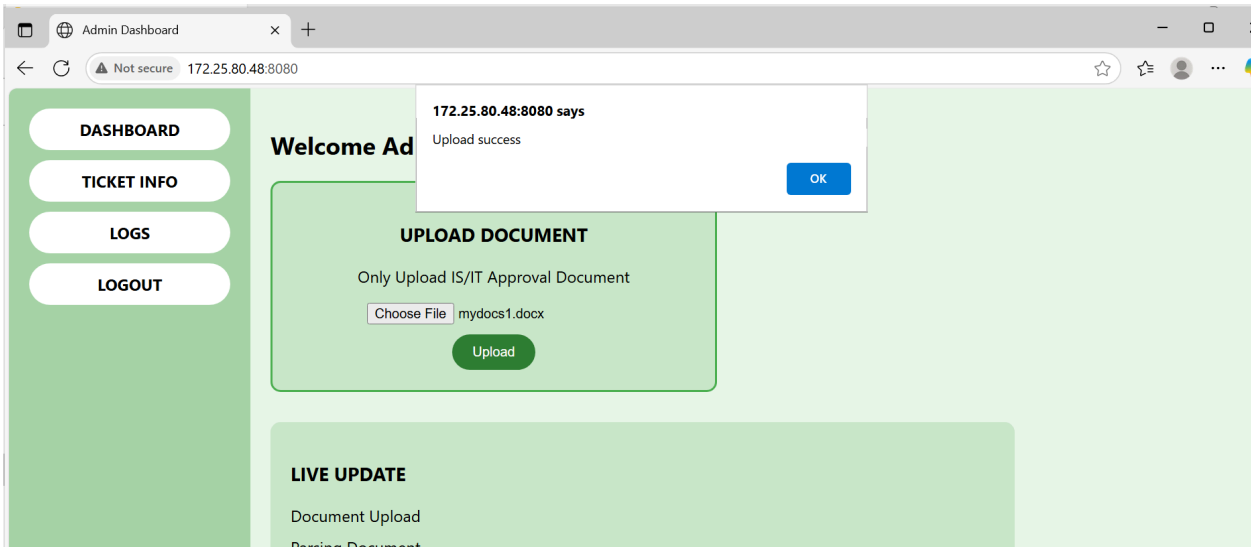


Fig. 3 Dashboard popup

Once the file upload is successful, the dashboard displays a popup to notify the user.

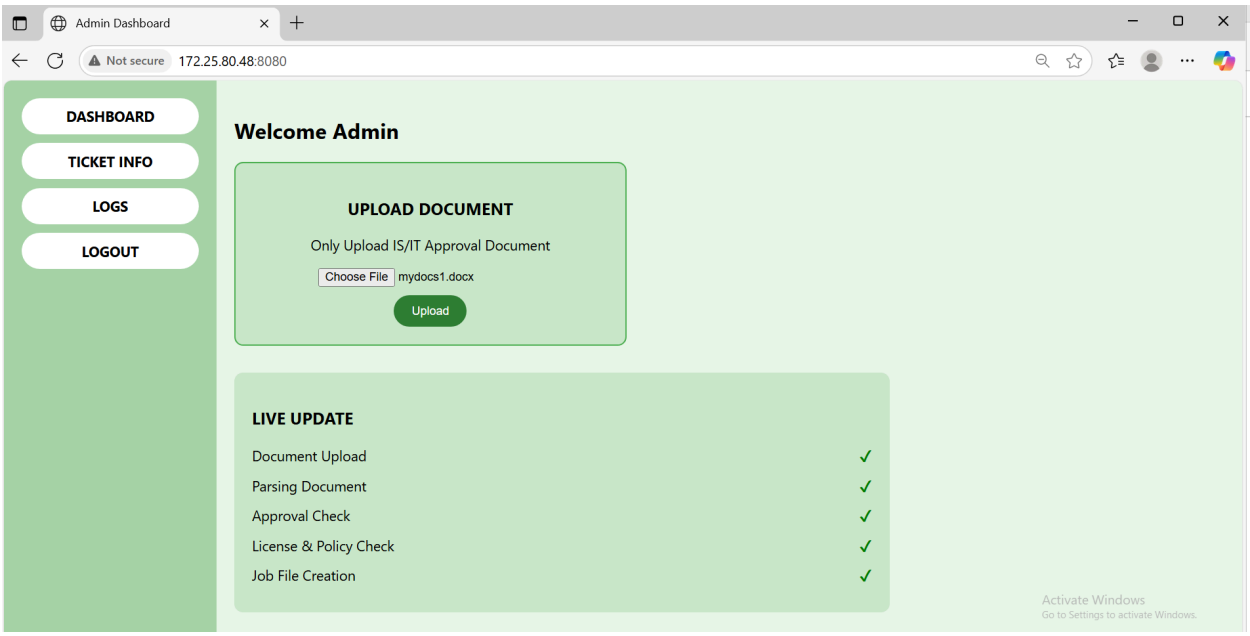


Fig. 4 Live update in action

As the Agent/Deepseek LLM, parses through the uploaded document and creates the job file, each step is updated on the website.

2.2. Logs

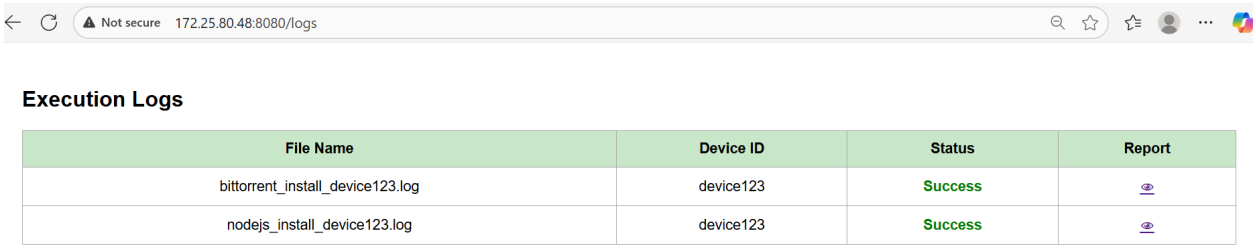


Fig. 5 Logs

The logs menu displays installation logs in a table format for easy tracking and debugging. An eye icon, under the Reports column, redirects to the respective log file for each record.

2.3. Ticket info

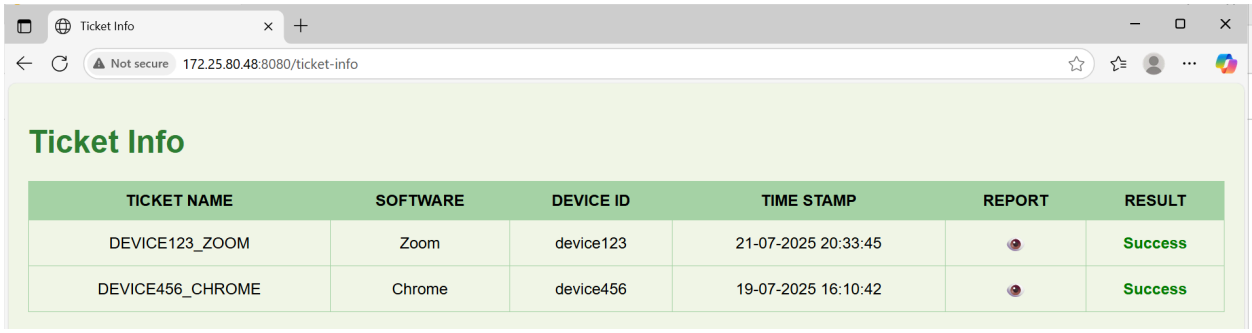


Fig. 6 Ticket Information


```
{
  "software": "Slack",
  "device_id": "device123",
  "hash": "unknown",
  "path": "\\\\172.25.80.10\\Share\\SlackInstaller.exe",
  "install_command": "cmd /c copy \"\\\\172.25.80.10\\Share\\SlackInstaller.exe\" \"%TEMP%\\SlackInstaller.exe\" && \"%TEMP%\\SlackInstaller.exe\" /s\"",
  "approved": true
}
```


The ticket information menu displays the status of job file creation. The eye icon here, also redirects to the job file created by the Agent/LLM.

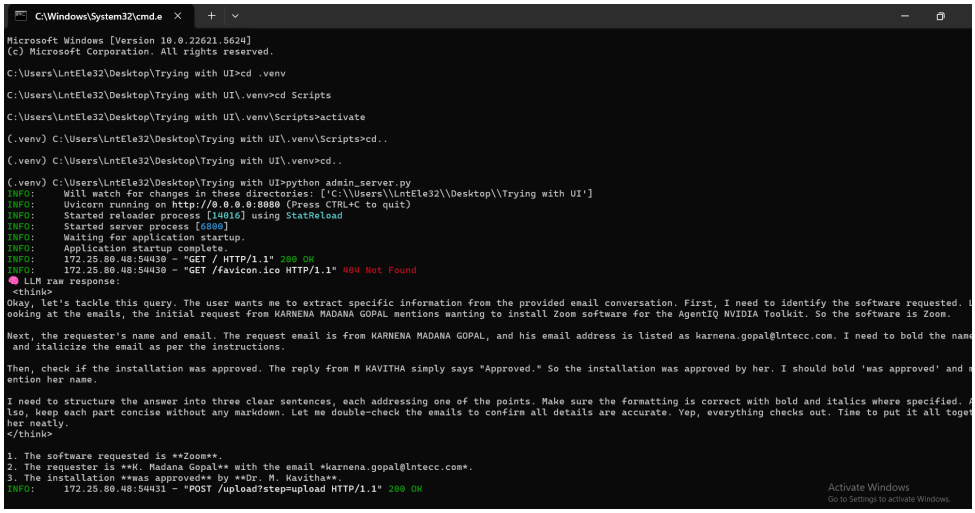


Fig. 7 CLI output

Fig. 7 shows the command line output. It includes the Agent's thinking process (in case of AgentIQ method), which is also reflected in the UI.

2.4. Client side notification

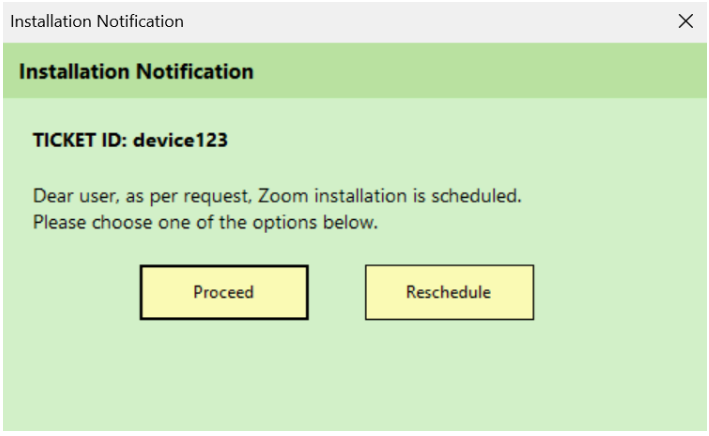


Fig. 8 Installation notification

Client agent repeatedly polls the Admin side server. Once a job file with device_id is found, the respective client agent notifies the client through a popup as shown in **Fig.**

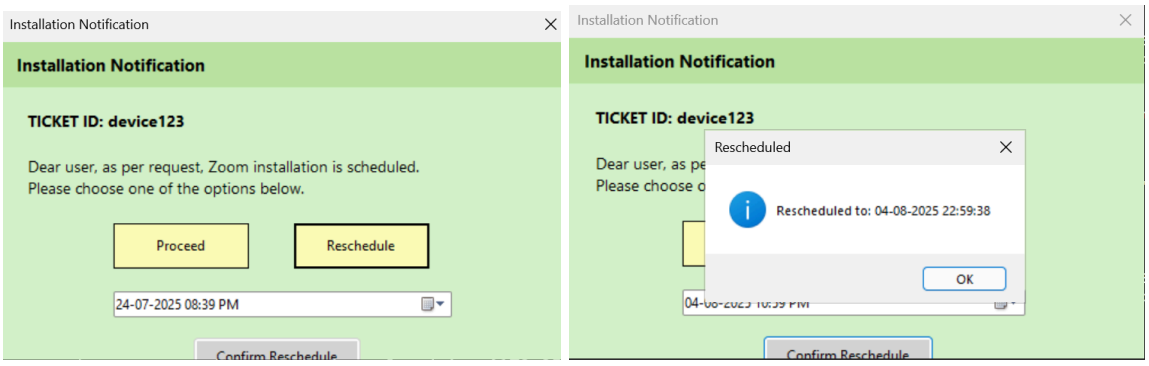


Fig. 9 Reschedule option

Once the client receives the popup, he/she has the option to either proceed with the installation right away or reschedule it to a later time. The above **Fig. 9** represents the reschedule option in action.

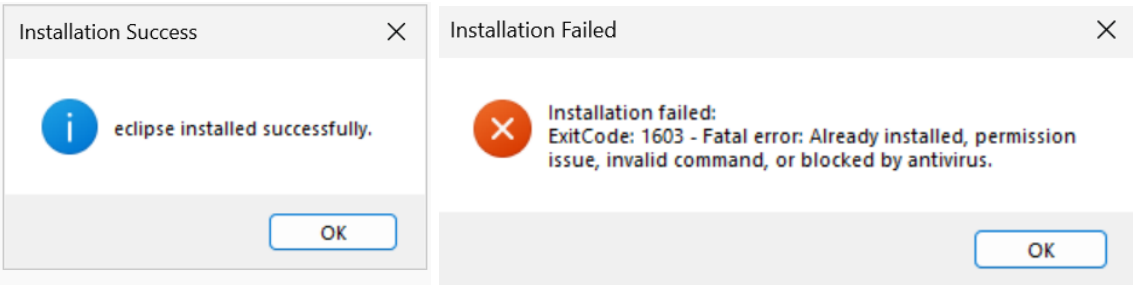


Fig. 10 Installation notification

In case of any success/ failure, the client is notified through a popup. The IT team will also be notified simultaneously via logs.

```
Ticket ID: device123
Software: Zoom
Client Mail ID: 123@gmail.com
Admin Name: Admin1
Job Sent At: 24-07-2025 19:52:04
Failure Detected At: 24-07-2025 19:52:09
Hash Verification: ✓ Verified
Installer File: ZoomInstaller.msi
Execution Status: Install failed
Error: ExitCode: 1603 - Fatal error: Already installed, permission issue, invalid command, or blocked by antivirus.
Log File: zoom_error_device123.log
```

Fig. 11 Log file

Fig. 11 displays the logs sent from client side to admin side.

3. Code Documentation

3.1.1 NVIDIA AgentIQ method - System Architecture

In this method, the admin machine leverages NVIDIA AgentIQ to parse IS (Information Security) approval emails. The extracted data includes software name, employee email, and approval intent, which are then validated against pre-existing lists as shown in **Fig. 12**

Flow:

- AgentIQ parses the approval email.
- Admin machine verifies the extracted software against:
    - software policy file
    - hashdb file for expected hashes
    - user-device ID mapped file
- A job file is generated and placed inside the jobs\ folder.
- The server hosts APIs for the client to poll job status.

- Installers are placed in a Shared Network Folder, accessible over LAN.

The client machine:

- Polls for job files
- Validates installer hash
- Executes installation and logs results to the server.

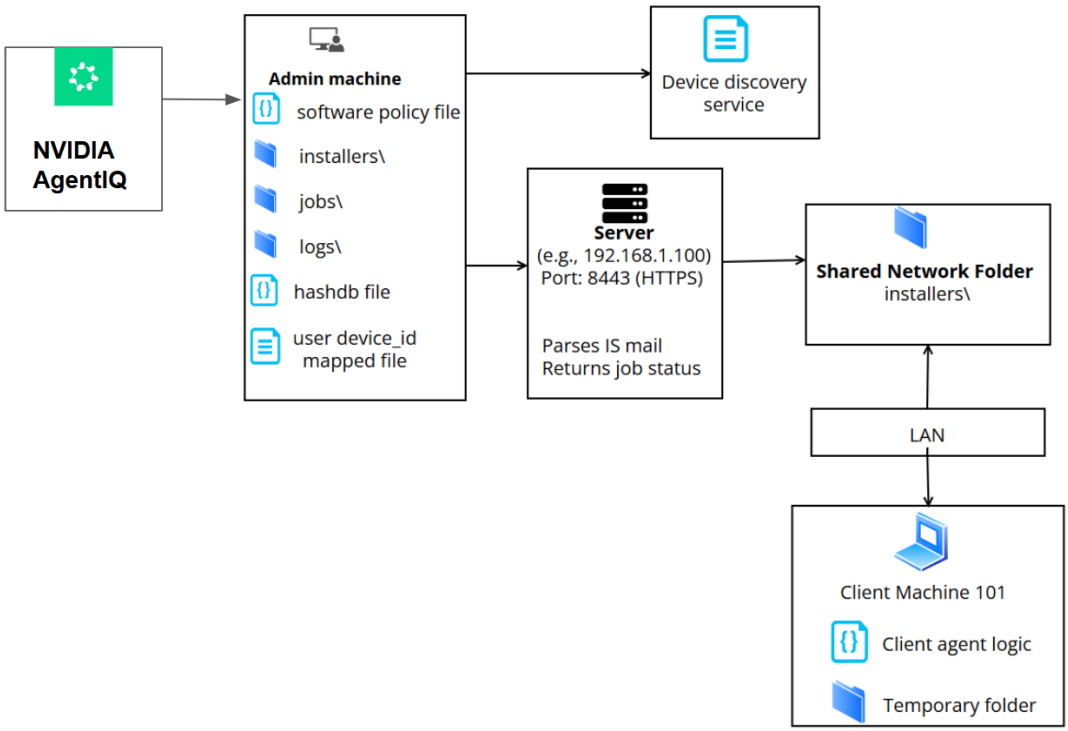


Fig. 12 NVIDIA AgentIQ - System Architecture

3.1.2 Deepseek method - System Architecture

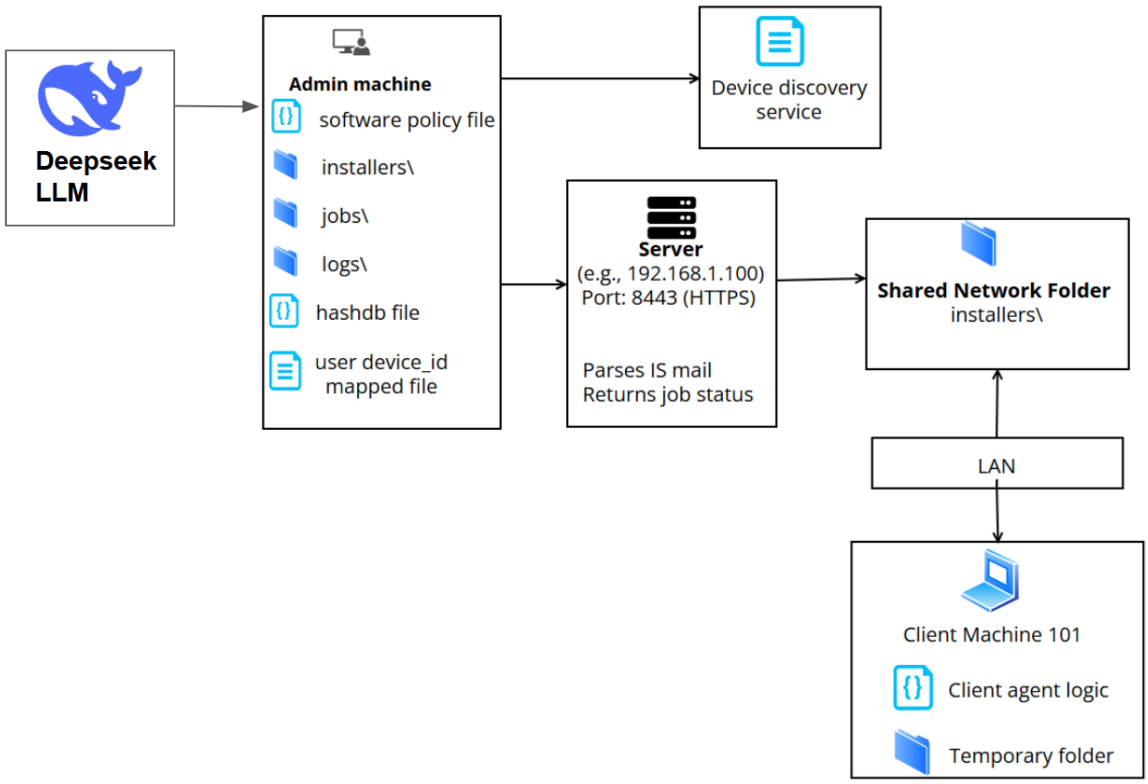


Fig. 13 Deepseek LLM - System Architecture

In **Fig. 13**, the admin system uses **DeepSeek**, a large language model, to interpret IS mail content with higher context understanding. This allows better handling of informal language or diverse mail templates.

Flow:

- DeepSeek LLM processes the mail via API or locally.
- It extracts entities and intents more flexibly than rule-based systems.

  The rest of the pipeline remains the same:
  - Data is validated
  - Job files are generated
  - Client installs software via LAN after validating hash
- The logging and installer paths are consistent across both methods.

3.1.3 Network Architecture

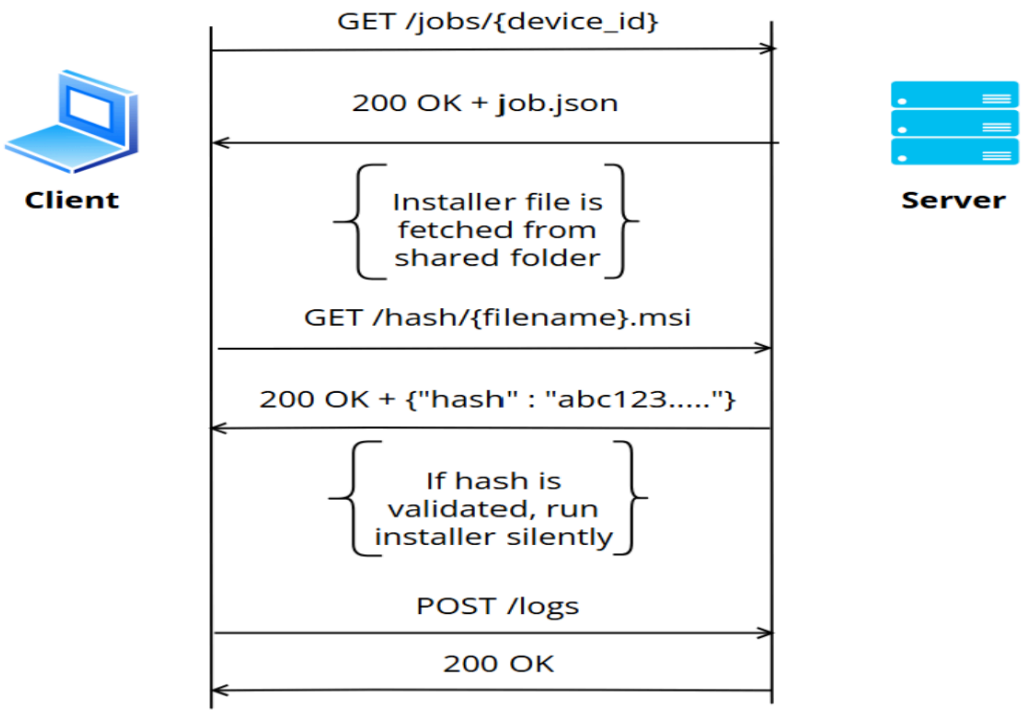
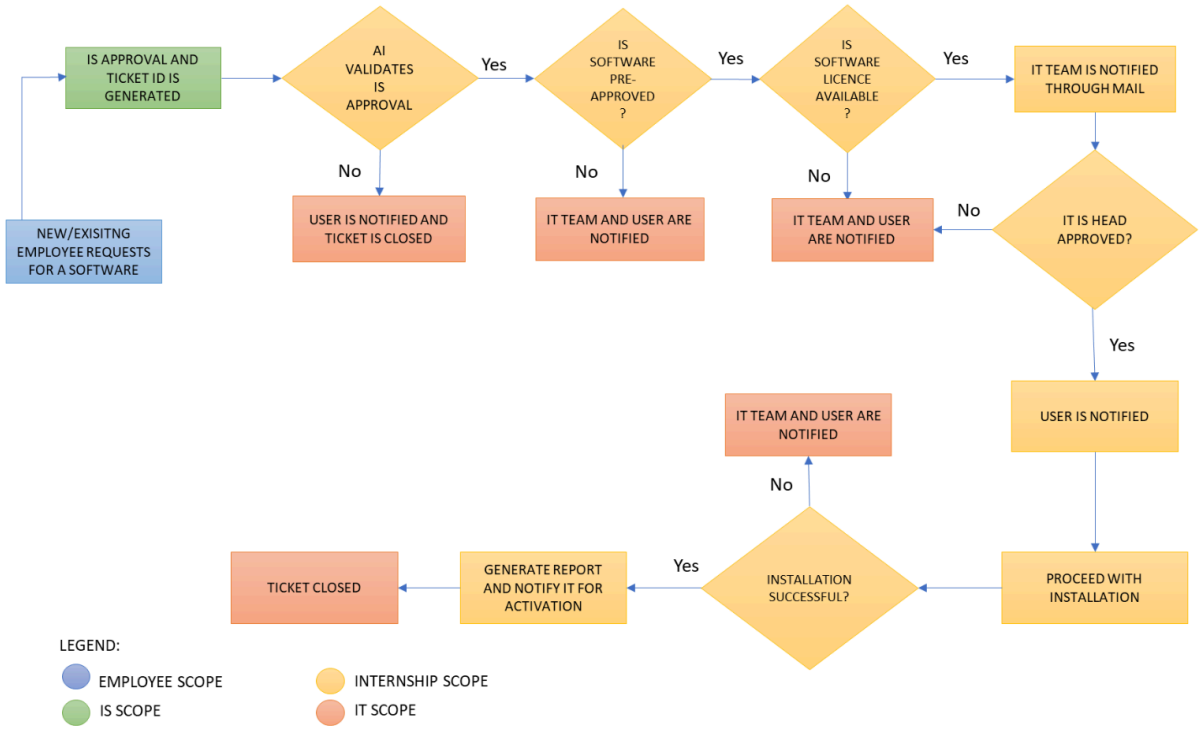


Fig. 14 Overall Network Architecture

Fig. 14 shows the overall network architecture depicting the client server communication.

3.1.4 Functional Flow



This section illustrates the step-by-step process for handling employee software requests, validation, and installation. The process is divided into three main scopes: Employee Scope, IT Scope, and Internship Scope.

Flow Description:

Employee Request Initiation:

A new or existing employee generates a software request ticket for a specific software, which is the Employee Scope.

Approval Validation by AI:

The AI validates whether the request contains a valid approval.

If no approval is found, the ticket is closed and both the employee and IT team are notified. This falls under the IT Scope.

Software Pre-Approval Check:

If the request is approved, the system checks if the software is pre-approved.

If not pre-approved, the IT team and user are notified, and the ticket is closed.

License Availability Check:

If the software is pre-approved, the system checks if the software license is available.

If no license is available, the IT team and user are notified, and the ticket is closed.

Head Approval:

If a valid license is available, the system verifies whether head approval is granted.

If no head approval, the IT team and user are notified, and the ticket is closed.

Ticket Closure or Proceeding to Activation:

If head approval is granted, a report is generated, and the IT team is notified for activation. This step falls under the Internship Scope.

Installation Success Check:

After activation, the system checks if the installation is successful.

If unsuccessful, the IT team and user are notified, and the ticket is closed.

Successful Installation:

If the installation is successful, the user is notified, and the system proceeds with the installation process.

3.2 NVIDIA AgentIQ - Source codes

Note: Make sure to follow the folder structure given below.

```
final/
├── _pycache_/
├── jobs/
├── logs/
├── server
├── static/
├── templates/
├── user_device_map
├── software_policy
├── installer_command.yaml
├── Hashdb.yaml
├── Software_name.yaml
nemo-agent-toolkit/
   examples/
```

```
software_approval_parser/
    ├── configs/
    ├── data/
    ├── scripts/
    ├── src/
    ├── .dockerignore
    ├── Dockerfile
    ├── README.md
```

Note: Ensure that these files are present in your working directory before proceeding.

Hashdb file:

```
ZoomInstaller.msi: "bbcf73741feaea00d215080d8cc01e833980cba2678da28346a93bf9ba964dde"
ChromeInstaller.msi: "ghi789jkl012"
FirefoxInstaller.msi: "mno345pqr678"
```

Software policy:

```
device123:
  allowed:
    - Zoom
    - Chrome
    - Microsoft cpp build tools
device456:
  allowed:
```

- Firefox
- Chrome

User_device_map:

karnena.gopal@lntecc.com:
device_id: device123
hari@gmail.com:
device_id: device456

Software_name:

Zoom: ZoomInstaller.msi
Google Chrome: ChromeSetup.msi
Slack: SlackInstaller.msi
Visual Studio Code: VSCodeSetup.msi
Bittorent: Bittorrent.exe
SumatraPDF: SumatraPDF.exe

Installer_command:

ZoomInstaller.msi: "cmd /c copy \"\\\\172.25.80.10\\Share\\ZoomInstaller.msi\" \"%TEMP%\\ZoomInstaller.msi\" && msiexec /i \"%TEMP%\\ZoomInstaller.msi\" /qn /norestart"
Bittorrent.exe: "cmd /c copy \"\\\\172.25.80.10\\Share\\Bittorrent.exe\" \"%TEMP%\\Bittorrent.exe\" && \"%TEMP%\\Bittorrent.exe\" /S"

Note: Clone this repository before you proceed:

https://github.com/NVIDIA/NeMo-Agent-Toolkit/tree/develop/examples/evaluation_and_profiling/email_phishing_analyzer All commands are given in the same repository.

3.2.1 Config.yaml

Path:

C:\NeMo-Agent-Toolkit\examples\software_approval_parser\configs\config.yaml

```
eval:
  general:
    output_dir: ./output
    console_output: true
    profiler:
      compute_llm_metrics: true
      workflow_runtime_forecast: true
      token_uniqueness_forecast: true
      csv_exclude_io_text: false
      prompt_caching_prefixes:
        enable: true
        min_frequency: 0.1
      bottleneck_analysis:
        enable_nested_stack: true
      concurrency_spike_analysis:
        enable: true
        spike_threshold: 7
  dataset:
    format: "json"
    path: ./input/email_data.json
  use_uvloop: true
  telemetry:
    logging:
      console:
        _type: console
        level: INFO
```

```
    file:
      _type: file
      path: /tmp/software_approval_parser.log
      level: DEBUG



functions:
  software_approval_parser:
    _type: software_approval_parser
    llm: nim_llm
    prompt: |
      Analyze the following IT approval mail. If the mail contains valid approval for software installation, extract the following:
      - software_name: (string) Name of the approved software
      - employee_email: (string) Email ID or Device ID (whichever is present)
      - is_approved: (boolean) True if approval is clearly granted
      - explanation: (string) Reasoning behind your extraction


      Mail content:
      {body}


      Return the result as a JSON object.
```

```
llms:
  nim_llm:
    _type: nim
    model_name: meta/llama-3.1-405b-instruct
    temperature: 0.0
    max_tokens: 512



workflow:
  _type: tool_calling_agent
  tool_names:
    - software_approval_parser
  llm_name: nim_llm
  verbose: true
  retry_parsing_errors: true
  max_retries: 3
  function_call_mode: always
```

Purpose:

This YAML configuration defines the setup for running the software_approval_parser using NVIDIA's NeMo Agent Toolkit (AgentIQ). It specifies the LLM model, function logic, dataset input, logging preferences, and workflow orchestration details for parsing IT approval emails.;

- **eval.general**

  - Controls general evaluation behavior including:
    - Output directory for results
    - Console logging
    - Profiler tools for LLM token analysis and runtime forecasting
    - Bottleneck and concurrency spike analysis for performance tuning

- **eval.dataset**

  - Specifies the format and path for the input data.
    - Here, a JSON file (./input/email_data.json) is used as the input dataset containing raw email text for analysis.

- **telemetry.logging**

  - Sets up both console and file logging.
    - Console logs are shown in real-time (level: INFO)
    - Debug logs are saved to /tmp/software_approval_parser.log

- **functions.software_approval_parser**

  - This defines the custom tool function to be executed by the agent.

- It uses an in-context prompt to instruct the LLM to extract:
  - software_name
  - employee_email or device ID
  - is_approved (True/False)
  - explanation of reasoning
- The function expects a mail body as input ({body}) and returns a structured JSON object.

llms.nim_llm

- Specifies the Large Language Model to be used:
  - Model: meta/llama-3.1-405b-instruct
  - Temperature: 0.0 for deterministic outputs
  - Token limit: 512

workflow

- This defines the agent logic:
  - Uses tool_calling_agent to invoke the parsing function
  - The nim_llm model is assigned to handle this workflow
  - Includes error handling (retry_parsing_errors: true) and retry logic (up to 3 retries)

3.2.2 Approval test file

Path:

data/approval_test.csv

```
subject,body,arrival_time,sender,intents,label,source,extra_info
Software Approval - Slack,"The installation of Slack is approved for hari@gmail.com by IS. Regards, IT Team",2025-07-01 10:30:00,itdesk@example.com,software,approved,is,ok
Approval: Zoom,"Hi Team, Zoom has been approved for device ID device456 by the IT supervisor.",2025-07-03 11:15:00,is_team@example.com,software,approved,is,ok
Re: Not Yet Approved,"Installation for Adobe is under review. Do not proceed.",2025-07-02 14:00:00,itmanager@example.com,software,rejected,is,warning
```

Purpose:

This CSV file contains sample IT approval emails to be used for testing the performance and accuracy of the email parsing system. It mimics real-world communication between employees and IT/IS teams, and is used as a **test dataset** to evaluate whether the LLM correctly extracts software names, approval status, and user/device information.

3.2.3 run_software_approval_e – Execution Script

Path:

scripts/run_software_approval_e

```
#!/bin/bash
HARDCODED_API_KEY="<YOUR_API_KEY_HERE>"
if [ ! -z "$1" ]; then
  export NVIDIA_API_KEY="$1"
  echo "Using API key from command line argument"
elif [ ! -z "$NVIDIA_API_KEY" ]; then
  echo "Using NVIDIA_API_KEY from environment"
else
  export NVIDIA_API_KEY="$HARDCODED_API_KEY"
  echo "Using hardcoded API key"
fi
CONFIG="configs/config.yml"
echo "Running software approval parser evaluation..."
aiq eval --config_file="$CONFIG"
```

Purpose:

This shell script serves as the **entry point** for executing the **email approval parser workflow** using NVIDIA AgentIQ. It handles **API key assignment**, selects the appropriate configuration file, and then triggers the aiq eval command for evaluation.

Functionality Breakdown:

- **API Key Handling:**

The script prioritizes API key sources in the following order:

1. **Command-line argument** (highest priority)
2. **Environment variable** NVIDIA_API_KEY (if already set)
3. **Hardcoded fallback key** (last resort)

This ensures flexibility when deploying in different environments — either securely passing keys at runtime or predefining them in the system.

```
if [ ! -z "$1" ]; then
  export NVIDIA_API_KEY="$1"
```

- **Configuration Binding:**

Sets the path to the YAML configuration file:

```
CONFIG="configs/config.yml"
```

This config file defines the prompt, function, model, and logging used during parsing.

- **Execution:**

Runs the **AgentIQ evaluation** using the defined configuration:

```
aiq eval --config_file="$CONFIG"
```

This command invokes the email parsing agent, passing the configuration for processing input data and returning extracted results.

3.2.4 software_approval_extractor – Rule-Based Email Parser

Path:

Software_approval_parser\src\aiq_software_approval_parser\tool\software_approval_extractor.py

```
from aiq.tool import register_tool
@register_tool
def software_approval_extractor(input: str) -> dict:
    import re
    software_match = re.search(r'installation of (\w+)', input, re.IGNORECASE)
    device_match = re.search(r'for (\S+@\S+)', input, re.IGNORECASE)
    approved_match = re.search(r'\bapproved\b', input, re.IGNORECASE)
    software = software_match.group(1) if software_match else None
    device_id = device_match.group(1) if device_match else None
    approval_status = "approved" if approved_match else "not approved"
    return {
        "software_name": software or "unknown",
        "device_id": device_id or "unknown",
        "approval_status": approval_status
    }
```

Purpose:

This Python function is a **custom tool registered with AgentIQ** to extract key approval details from unstructured email content using simple **regex-based logic**. It serves as an alternative to LLM-based parsing or can be used for comparison/baseline testing.

Function: software_approval_extractor(input: str) → dict

Registered Tool:

The function is decorated with @register_tool, which allows it to be recognized and invoked by the **AgentIQ tool-calling agent** defined in the YAML workflow.

Input:

- input – a string representing the **raw email body**

Output:

- Returns a dictionary with extracted values:

```
{
  "software_name": ...,
   "device_id": ...,
  "approval_status": ...
}
```

3.2.5 Register.py

Path:

software_approval_parser\src\aiq_software_approval_parser\register.py

```
import json
import logging
from typing import Any
from aiq.builder.builder import Builder
from aiq.builder.framework_enum import LLMFrameworkEnum
from aiq.builder.function_info import FunctionInfo
from aiq.cli.register_workflow import register_function
from aiq.data_models.component_ref import LLMRef
from aiq.data_models.function import FunctionBaseConfig
from aiq_software_approval_parser.utils import smart_parse
logger = logging.getLogger(__name__)
class SoftwareApprovalParserConfig(FunctionBaseConfig,
name="software_approval_parser"):
    _type: str = "software_approval_parser"
    llm: LLMRef
    prompt: str
@register_function(config_type=SoftwareApprovalParserConfig,
framework_wrappers=[LLMFrameworkEnum.LANGCHAIN])
async def software_approval_parser(config: SoftwareApprovalParserConfig, builder:
Builder) -> Any:
    async def _parse_software_approval(text: str) -> str:
        llm = await builder.get_llm(llm_name=config.llm,
wrapper_type=LLMFrameworkEnum.LANGCHAIN)
        response = await llm.apredict(config.prompt.format(body=text))
        try:
            result = smart_parse(response)
```

```
            return json.dumps({
                "software_name": result.get("software_name"),
                "employee_email": result.get("employee_email"),
                "is_approved": result.get("is_approved", False),
                "explanation": result.get("explanation", "No explanation found.")
            })
        except json.JSONDecodeError:
            return "Error: Could not parse LLM response as JSON"
    yield FunctionInfo.from_fn(
    _parse_software_approval,
    description="Extract software installation approval details from IT emails."
)
```

Purpose:

This script registers the software_approval_parser **function** with AgentIQ, defining how it should be executed using a specified LLM framework. It acts as the bridge between the prompt template, model execution, and result handling.

- SoftwareApprovalParserConfig
  - Inherits from FunctionBaseConfig
  - Defines the schema for this tool:
    - llm: LLM model reference
    - prompt: the in-context prompt template used to extract required fields

class SoftwareApprovalParserConfig(FunctionBaseConfig, name="software_approval_parser"):

_type: str = "software_approval_parser"

llm: LLMRef

prompt: str

- @register_function(...)
  - Registers software_approval_parser as a callable tool in the AgentIQ framework
  - Supports LANGCHAIN as the LLM framework wrapper

Function**:** software_approval_parser(...)

- Takes in the registered config and the AgentIQ Builder object
- Defines an **async internal function** _parse_software_approval(text)
  - Uses builder.get_llm(...) to load the model (e.g., LLaMA-3)
  - Applies the provided prompt to the raw email text
  - Asynchronously retrieves a response from the LLM

- **Parsing the LLM Response**
  - Uses the custom utility smart_parse() to parse the LLM's output into structured fields:
    - software_name
    - employee_email

- Is_approved
- explanation

If parsing fails, a fallback error message is returned:

- "Error: Could not parse LLM response as JSON"

3.2.6 utils.py – Smart Response Parser

Path:

software_approval_parser\src\aiq_software_approval_parser\utils.py

```
# SPDX-FileCopyrightText: Copyright (c) 2024-2025, NVIDIA CORPORATION &
AFFILIATES. All rights reserved.
# SPDX-License-Identifier: Apache-2.0
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
# http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
import json
import re
def smart_parse(text: str) -> dict:
    """
    Smart parser that attempts to extract structured data from a string using multiple approaches.
    Handles:
    1. Pure JSON objects
    2. JSON embedded in text
    3. Key-value pairs in formats like:
       - key="value"
```

```
   - key=value
   - Key: "value"
   - key: value
4. Plain text (stored under 'message' key)
Args:
   text (str): Input text to parse
Returns:
   dict: Parsed data or {'message': text} if no structure found
"""
# First try: Parse as pure JSON
try:
   return json.loads(text)
except json.JSONDecodeError:
   # Second try: Look for JSON within text
   json_match = re.search(r'{.*}', text, re.DOTALL)
   if json_match:
      try:
         return json.loads(json_match.group(0))
      except json.JSONDecodeError:
         pass
   # Third try: Parse key-value pairs
   pattern = re.findall(
      r'(\w+)=["\']([^"\']+)["\']|'  # key="value"
      r'(\w+)=([\w.]+)|'  # key=value
      r'(\w+):\s*["\']([^"\']+)["\']|'  # Key: "value"
      r'(\w+):\s*([\w.]+)',  # key: value
```

```
            text)
        if pattern:
            parsed_data = {}
            remaining_str = text
            for match in pattern:
                key = next(m for m in [match[0], match[2], match[4], match[6]] if m)
                value = next(m for m in [match[1], match[3], match[5], match[7]] if m)
                parsed_data[key.lower()] = value
                # Remove matched text from remaining string
                for possible_format in [f'{key}={value}', f'{key}: {value}', f'{key}="{value}"',
f'{key}: "{value}"']:
                    remaining_str = remaining_str.replace(possible_format, '')
            # Add remaining text as message if it exists
            remaining_str = remaining_str.strip().strip(',').strip()
            if remaining_str:
                parsed_data['message'] = remaining_str
            return parsed_data
        # Fallback: Return plain text as message
        return {'message': text}
```

Purpose:

This utility module defines the function smart_parse(), which is responsible for extracting structured data from LLM responses or raw text. Since LLM outputs can vary in formatting, this parser applies multiple strategies to reliably extract key fields like software name, email, approval status, etc.

It acts as a resilient fallback mechanism in the event of:

Slightly malformed JSON

Embedded structured data

Loose key-value pairs

Plain natural language responses

Function: smart_parse(text: str) → dict

Input:

A string response from an LLM or text body to be parsed.

Output:

A dictionary of extracted data.

If parsing fails, returns:

{'message': original_text}

3.2.7 pyproject.toml

Path:

software_approval_parser/pyproject.toml

```
[build-system]
build-backend = "setuptools.build_meta"
requires = ["setuptools >= 64", "setuptools-scm>=8"]

[tool.setuptools_scm]
root = "../.."

[project]
name = "aiq_software_approval_parser"
dynamic = ["version"]
dependencies = [
  "aiqtoolkit[langchain]",
  "arize-phoenix==6.1.*",
  "bs4==0.0.2",
  "networkx~=3.4",
  "openinference-instrumentation-langchain==0.1.29",
]
requires-python = ">=3.11,<3.13"
description = "Simple software approval parser AIQ toolkit example"
keywords = ["ai", "approval", "agents"]
classifiers = ["Programming Language :: Python"]
```

```
[tool.uv.sources]
aiqtoolkit = { path = "../../", editable = true }


[project.entry-points.'aiq.components']
aiq_software_approval_parser = "aiq_software_approval_parser.register"
```

Purpose:

This file defines the **build system, project metadata, dependencies, and entry points** for the aiq_software_approval_parser module. It is essential for packaging the project, managing dependencies, and enabling its integration with the AgentIQ runtime.

- **[build-system]**

Specifies the Python build backend and required tools:

build-backend = "setuptools.build_meta"

requires = ["setuptools >= 64", "setuptools-scm >= 8"]

- Uses setuptools to build the package
- setuptools-scm is used for automatic versioning based on Git tags

- **[project]**

Defines metadata for the Python package:

name = "aiq_software_approval_parser"

description = "Simple software approval parser AIQ toolkit example"

keywords = ["ai", "approval", "agents"]

- dynamic = ["version"] indicates that the version is inferred using setuptools-scm
- Compatible with Python >=3.11 and <3.13
- Declares required dependencies to run the module

- dependencies

Includes essential libraries:

```
[
  "aiqtoolkit[langchain]",              # Core AgentIQ tools with LangChain support
  "arize-phoenix==6.1.*",               # Observability and metrics logging
  "bs4==0.0.2",                         # BeautifulSoup for HTML parsing (if needed)
  "networkx~=3.4",                      # Graph-based analysis tools
  "openinference-instrumentation-langchain==0.1.29" # For observability in LangChain
]
```

- **[tool.setuptools_scm]**

Specifies that the version should be derived from the Git root two levels up:

root = "../.."

- **[tool.uv.sources]**

Used when working with **editable local packages**. This allows for local development by pointing to the relative path:

aiqtoolkit = { path = "../../", editable = true }

- **[project.entry-points.'aiq.components']**

This entry point tells AgentIQ to register your component:

aiq_software_approval_parser = "aiq_software_approval_parser.register"

- Maps the component name to the register.py file where the tool is defined and exposed.

3.2.8 server.py

Path:

server.py

```
import os
import re
import json
import yaml
import tempfile
import subprocess
from fastapi import FastAPI, UploadFile, File, Request
from fastapi.responses import HTMLResponse, FileResponse, PlainTextResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from datetime import datetime
from fastapi.responses import JSONResponse
# Setup
app = FastAPI(docs_url=None, redoc_url=None)
CONFIG_FILE =
r"C:\NeMo-Agent-Toolkit\examples\software_approval_parser\configs\config.yml"
app.mount("/static", StaticFiles(directory="static"), name="static")
templates = Jinja2Templates(directory="templates")
```

```
@app.get("/", response_class=HTMLResponse)
async def index():
    return FileResponse("static/index.html")
@app.get("/ticket-info", response_class=HTMLResponse)
async def ticket_info(request: Request):
    jobs = []
    if os.path.exists("jobs"):
        for fname in os.listdir("jobs"):
            if fname.endswith(".json"):
                with open(os.path.join("jobs", fname)) as f:
                    job = json.load(f)
                    jobs.append({
                        "ticket_id": fname.replace("job_", "").replace(".json", "").upper(),
                        "software": job.get("software"),
                        "device_id": job.get("device_id"),
                        "status": "APPROVED" if job.get("approved") else "REJECTED",
                        "result": "Success" if job.get("approved") else "Failed",
                        "file": fname,
                        "timestamp": datetime.fromtimestamp(os.path.getmtime(os.path.join("jobs",
fname))).strftime("%d-%m-%Y %H:%M:%S")
                    })
    return templates.TemplateResponse("ticket_info.html", {"request": request, "jobs": jobs})
@app.get("/logs", response_class=HTMLResponse)
async def logs_view(request: Request):
    logs = []
    log_dir = "logs"
```

```
    if os.path.exists(log_dir):
        for fname in os.listdir(log_dir):
            path = os.path.join(log_dir, fname)
            if fname.endswith(".log") and os.path.isfile(path):
                with open(path, "r", encoding="utf-8") as f:
                    content = f.read()
                    # Extract device_id and status
                    device_match = re.search(r"Ticket ID:\s*(.*)", content)
                    status_match = re.search(r"Execution Status:\s*(.*)", content)
                    device_id = device_match.group(1).strip() if device_match else "unknown"
                    status_text = status_match.group(1).strip().lower() if status_match else ""
                    status = "Success" if "success" in status_text else "Failed"
                    logs.append({
                        "filename": fname,
                        "device_id": device_id,
                        "status": status,
                    })
    return templates.TemplateResponse("logs.html", {"request": request, "logs": logs})
@app.get("/view-log/{filename}", response_class=PlainTextResponse)
async def view_log(filename: str):
    path = os.path.join("logs", filename)
    if not os.path.exists(path):
        return PlainTextResponse("File not found", status_code=404)
    with open(path, "r", encoding="utf-8") as f:
        return PlainTextResponse(f.read())
```

```
@app.get("/view-job/{filename}", response_class=PlainTextResponse)
async def view_job(filename: str):
    path = os.path.join("jobs", filename)
    if not os.path.exists(path):
        return PlainTextResponse(content="File not found", status_code=404)
    with open(path) as f:
        return PlainTextResponse(f.read())
from fastapi.templating import Jinja2Templates
templates = Jinja2Templates(directory="templates")
# --- Save uploaded file temporarily ---
def save_temp_file(uploaded_file: UploadFile) -> str:
    suffix = os.path.splitext(uploaded_file.filename)[-1]
    with tempfile.NamedTemporaryFile(delete=False, suffix=suffix, mode="wb") as tmp:
        tmp.write(uploaded_file.file.read())
        return tmp.name
# --- Extract text from uploaded file ---
def extract_text(path: str) -> str:
    if path.endswith(".pdf"):
        from PyPDF2 import PdfReader
        reader = PdfReader(path)
        return "\n".join(page.extract_text() or "" for page in reader.pages)
    elif path.endswith(".docx"):
        from docx import Document
        doc = Document(path)
        return "\n".join(p.text for p in doc.paragraphs)
    elif path.endswith(".txt"):
```

```
        with open(path, "r", encoding="utf-8") as f:
            return f.read()
    return ""
# --- Run AIQ with extracted text ---
def run_aiq_with_input(text: str) -> str:
    result = subprocess.run(
        [r"C:\NeMo-Agent-Toolkit\.venv\Scripts\aiq.exe", "run", "--config_file", CONFIG_FILE,
"--input", text],
        capture_output=True,
        text=True
    )
    return result.stdout + "\n" + result.stderr
# --- Extract JSON string from tool's response in raw logs ---
def extract_tool_response_from_log(output: str) -> dict:
    matches = re.findall(r'"software_name":.*?\}', output, re.DOTALL)
    if matches:
        json_str = "{" + matches[0]
        try:
            return json.loads(json_str)
        except json.JSONDecodeError:
            return {}
    return {}
# --- Load helper mappings from YAML ---
def load_yaml_mapping(filename: str) -> dict:
    with open(filename, "r") as f:
        return yaml.safe_load(f)
```

```
# --- Upload endpoint ---
# --- Upload endpoint ---
@app.post("/upload-email/")
async def upload_email(file: UploadFile = File(...)):
    try:
        temp_path = save_temp_file(file)
        content = extract_text(temp_path)
        aiq_output = run_aiq_with_input(content)
        tool_response = extract_tool_response_from_log(aiq_output)
        software = tool_response.get("software_name", "unknown")
        email = tool_response.get("employee_email", "unknown")
        approved = tool_response.get("is_approved", False)
        explanation = tool_response.get("explanation", "No explanation found.")
        # Load mappings
        user_map = load_yaml_mapping("user_device_map.yaml")
        hash_map = load_yaml_mapping("hashdb.yaml")
        policy_map = load_yaml_mapping("software_policy.yaml")
        command_map = load_yaml_mapping("installer_command.yaml")
        device_id = user_map.get(email, "unknown_device")
        installer_filename = policy_map.get(software, None)
        if installer_filename:
            installer_path = f"\\\\172.25.80.10\\Share\\{installer_filename}"
            file_hash = hash_map.get(installer_filename, "unknown_hash")
        else:
            installer_path = "unknown_path"
            file_hash = "unknown_hash"
```

```
install_command = command_map.get(installer_filename, "command_not_found")
job_data = {
    "software": software,
    "device_id": device_id,
    "hash": file_hash,
    "path": installer_path,
    "install_command": install_command,
    "approved": approved
}
os.makedirs("jobs", exist_ok=True)
os.makedirs("log", exist_ok=True)
safe_email = email.replace("@", "_").replace(".", "_")
job_filename = f"job_{device_id}_{software}.json"
log_filename = f"log_{software}_{safe_email}.json"
job_path = os.path.join("jobs", job_filename)
log_path = os.path.join("log", log_filename)
with open(job_path, "w") as f:
    json.dump(job_data, f, indent=4)
with open(log_path, "w") as f:
    json.dump(job_data, f, indent=4)
# ✅ Restore raw output in response
response_text = (
    "\n========= Uploaded Content =========\n"
    f"{content.strip()}\n\n"
    "========= AgentIQ Output =========\n"
```

```
            f"{aiq_output.strip()}\n\n"
            "========= Extracted Fields =========\n"
            f"Software: {software}\n"
            f"Email: {email}\n"
            f"Approved: {approved}\n"
            f"Explanation: {explanation}\n"
            f"Device ID: {device_id}\n"
            f"Installer Path: {installer_path}\n"
            f"File Hash: {file_hash}\n"
            f"Install Command: {install_command}\n\n"
            f"Job saved to: {job_path}\n"
            f"Log saved to: {log_path}"
        )
        return PlainTextResponse(content=response_text)
    except Exception as e:
        return PlainTextResponse(content=f"Error: {str(e)}", status_code=500)
@app.get("/fetch-job/{device_id}", response_class=JSONResponse)
async def fetch_job(device_id: str):
    job_dir = "jobs"
    print(f"🔍 Looking for jobs for device_id: {device_id}")

    if not os.path.exists(job_dir):
        print("❌ Jobs folder does not exist")
        return JSONResponse(content={"error": "Jobs folder does not exist."}, status_code=404)
```

```
    for fname in os.listdir(job_dir):
        print(f"📁 Checking file: {fname}")
        if fname.startswith(f"job_{device_id}_") and fname.endswith(".json"):
            path = os.path.join(job_dir, fname)
            try:
                print(f"✅ Found match: {fname}")
                with open(path, "r") as f:
                    job_data = json.load(f)
                os.remove(path)  # Delete after reading
                return JSONResponse(content=job_data)
            except Exception as e:
                print("❌ Failed to read job:", e)
                return JSONResponse(content={"error": f"Failed to read job: {str(e)}"},
status_code=500)
    print("⚠️ No matching job found.")
    return JSONResponse(content={"message": "No job found for the given device ID."},
status_code=404)
@app.get("/get_job", response_class=JSONResponse)
async def get_job_by_query(device_id: str):
    return await fetch_job(device_id)
@app.post("/send_log", response_class=JSONResponse)
async def receive_log(request: Request):
    logs_dir = "logs"
    os.makedirs(logs_dir, exist_ok=True)
```

```
try:
    raw_body = await request.body()
    text_content = raw_body.decode("utf-8").strip()
    # ✅ Optional: Parse ticket ID or device ID from the content to name the file
    ticket_id_match = re.search(r'Ticket ID:\s*(\S+)', text_content)
    device_id = ticket_id_match.group(1) if ticket_id_match else "unknown"
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"{device_id}_{timestamp}.txt"
    log_path = os.path.join(logs_dir, filename)
    # ✅ Save raw log content to a text file
    with open(log_path, "w", encoding="utf-8") as f:
        f.write(text_content)
    print(f"✅ Log stored as: {filename}")
    return JSONResponse(content={"message": f"Log saved as {filename}."})
except Exception as e:
    print(f"❌ Failed to save log: {e}")
    return JSONResponse(content={"error": str(e)}, status_code=500)
```

Purpose:

This script serves as the **central controller** of the admin-side backend. It provides an interface for:

- Uploading email approvals

- Parsing approvals using AgentIQ
- Generating job files and logs
- Serving dashboards, logs, and job data
- Rendering HTML pages using Jinja2 templates

Execution Command:

```
uvicorn server:app --host 0.0.0.0 --port 8000
```

3.3. Deepseek LLM Method

Folder structure:

```
Deepseek_method/
├── _pycache_/
├── jobs/
├── processed_pdfs/
├── reports/
├── received_logs/
├── static/
├── templates/
├── uploads/
├── .venv/
├── software_policy.yaml
├── hashdb.yaml
├── admin_server.py
├── software_name.yaml
├── installer_command.yaml
├── user_device_map.yaml
├── hash_creation.py
```

3.3.1 Admin_server.py

```
from fastapi import FastAPI, File, UploadFile, Query, Request
from fastapi.responses import HTMLResponse, JSONResponse, PlainTextResponse
from fastapi.staticfiles import StaticFiles
from openai import OpenAI
import os, json, re, shutil, time
from docx import Document
from PyPDF2 import PdfReader
from pathlib import Path
import yaml
from datetime import datetime
from fastapi.templating import Jinja2Templates
app = FastAPI()
# Mount UI and jobs folder for file access
app.mount("/static", StaticFiles(directory="static"), name="static")
app.mount("/jobs", StaticFiles(directory="jobs"), name="jobs")
# DeepSeek (OpenAI-compatible) client
client = OpenAI(
    base_url="https://integrate.api.nvidia.com/v1",

api_key="nvapi-4FnNQLeWm_Nc91_8C4EsM9Ph0d0WQe_-Mib79h1EF4kfao_C7RRLGaq
URbN2f0D7"
)
# Ensure folders exist
for folder in ["uploads", "jobs", "reports", "received_logs"]:
```

```
    os.makedirs(folder, exist_ok=True)
# Generate default YAMLs if missing
config_files = {
    "user_device_map.yaml": """
user1:
  device_id: device123
user2:
  device_id: device456
""",
    "software_policy.yaml": """
device123:
  allowed:
    - Zoom
    - Chrome
device456:
  allowed:
    - Firefox
""",
    "hashdb.yaml": """
ZoomInstaller.msi: "abc123def456"
ChromeInstaller.msi: "ghi789jkl012"
FirefoxInstaller.msi: "mno345pqr678"
"""
}
for fname, content in config_files.items():
    if not os.path.exists(fname):
```

```
        with open(fname, "w") as f:
            f.write(content.strip())
@app.get("/", response_class=HTMLResponse)
async def index():
    try:
        return Path("static/index.html").read_text(encoding="utf-8")
    except Exception:
        # Friendly fallback if UI is missing
        return HTMLResponse(
            "<h1>Admin Software Installer</h1><p>Home page not found. Please add
<code>static/index.html</code>.</p>",
            status_code=200,
        )


@app.post("/upload")
async def upload_file(step: str = Query(...), file: UploadFile = File(...)):
    explanation = []
    text = ""
    # Log incoming query param and filename
    print(f"[DEBUG] Received /upload request with step='{step}', filename='{file.filename}'")
    # Save uploaded file
    try:
        temp_path = Path("uploads")
        temp_path.mkdir(parents=True, exist_ok=True)
        file_location = temp_path / file.filename
        with open(file_location, "wb") as f:
```

```
        shutil.copyfileobj(file.file, f)
    explanation.append("✅ File saved successfully")
    print(f"[DEBUG] File saved at: {file_location}")
except Exception as e:
    explanation.append(f"❌ Failed to save file: {str(e)}")
    print(f"[ERROR] Failed to save uploaded file: {e}")
    print(f"[DEBUG] Explanation: {explanation}")
    return JSONResponse({"status": "failed", "explanation": explanation}, status_code=400)


# Extract text based on file extension
try:
    ext = file.filename.lower().split(".")[-1]
    print(f"[DEBUG] File extension detected: .{ext}")
    if ext == "docx":
        doc = Document(file_location)
        text = "\n".join([p.text for p in doc.paragraphs])
        explanation.append("✅ Extracted text from .docx")
        print(f"[DEBUG] Extracted text length from .docx: {len(text)} characters")
    elif ext == "txt":
        with open(file_location, "r", encoding="utf-8") as f:
            text = f.read()
        explanation.append("✅ Extracted text from .txt")
        print(f"[DEBUG] Extracted text length from .txt: {len(text)} characters")
    elif ext == "pdf":
        reader = PdfReader(str(file_location))
        text = "\n".join(page.extract_text() or "" for page in reader.pages)
```

```
            explanation.append("✅ Extracted text from .pdf")
            print(f"[DEBUG] Extracted text length from .pdf: {len(text)} characters")
        elif ext == "doc":
            explanation.append("❌ .doc format not supported")
            print(f"[WARN] Unsupported file type: .doc")
            print(f"[DEBUG] Explanation: {explanation}")
            return JSONResponse({"status": "failed", "explanation": explanation},
status_code=400)
        else:
            explanation.append(f"❌ Unsupported file type: .{ext}")
            print(f"[WARN] Unsupported file type: .{ext}")
            print(f"[DEBUG] Explanation: {explanation}")
            return JSONResponse({"status": "failed", "explanation": explanation},
status_code=400)
    except Exception as e:
        explanation.append(f"❌ Failed to extract text: {str(e)}")
        print(f"[ERROR] Exception during text extraction: {e}")
        print(f"[DEBUG] Explanation: {explanation}")
        return JSONResponse({"status": "failed", "explanation": explanation}, status_code=400)
    try:
        print("[DEBUG] Calling OpenAI DeepSeek model...")
        start_time = time.time()
        fields = extract_fields_with_deepseek(text)
        end_time = time.time()
        print(f"[DEBUG] OpenAI API call duration: {end_time - start_time:.2f} seconds")
        explanation.append(f"✅ Fields extracted: {fields}")
```

```
        print(f"[DEBUG] Extracted fields: {fields}")
        if not fields.get("approved"):
            explanation.append("❌ Installation not approved")
            save_report(fields, explanation)
            print(f"[INFO] Installation not approved, returning failure.")
            return JSONResponse({"status": "failed", "explanation": explanation},
status_code=400)
        job_file = generate_job_file(fields)
        explanation.append(f"✅ Job file created: {job_file}")
        save_report(fields, explanation)
        print(f"[DEBUG] Job file created: {job_file}")
        return {"status": "success", "job_file": f"/jobs/{job_file}", "explanation": explanation}
    except Exception as e:
        explanation.append(f"❌ Exception occurred: {str(e)}")
        save_report({}, explanation)
        print(f"[ERROR] Exception during field extraction or job creation: {e}")
        print(f"[DEBUG] Explanation: {explanation}")
        return JSONResponse({"status": "failed", "explanation": explanation}, status_code=400)
@app.post("/send_log")
async def receive_log(logfile: UploadFile = File(...)):
    log_path = os.path.join("received_logs", logfile.filename)
    with open(log_path, "wb") as f:
        f.write(await logfile.read())
    print(f"[📥] Log received: {log_path}")
    return {"status": "Log received"}
@app.get("/get_job")
```

```
def get_job(device_id: str = Query(...)):
    for fname in os.listdir("jobs"):
        if fname.startswith(f"job_{device_id}_") and fname.endswith(".json"):
            job_path = os.path.join("jobs", fname)
            with open(job_path) as f:
                job_data = json.load(f)
            os.remove(job_path)
            return job_data
    return JSONResponse({"error": "No job for this device"}, status_code=404)
@app.get("/get_report")
def get_report(device_id: str, software: str):
    report_file = f"reports/report_{device_id}_{software.lower()}.json"
    if os.path.exists(report_file):
        with open(report_file) as f:
            return json.load(f)
    return JSONResponse({"error": "Report not found"}, status_code=404)
# ---------- UTILITIES ----------
def get_device_id_from_email(email: str):
    username = email.split("@")[0]
    with open("user_device_map.yaml") as f:
        device_map = yaml.safe_load(f)
    for user, details in device_map.items():
        if username.lower() in user.lower():
            return details["device_id"]
    return username
def extract_fields_with_deepseek(text: str):
```

```
    prompt = (
        "You are a JSON-only generator. Do NOT explain your answer.\n"
        "From the following email thread, extract and return ONLY the JSON object in this format:\n"
        "{\n"
        '  "software": "software_name",\n'
        '  "requester_name": "requester_name",\n'
        '  "email": "requester_mail",\n'
        '  "approved": true or false,\n'
        '  "approver": "approver_name"\n'
        "}\n\n"
        "Do NOT include any explanation or text outside the JSON. Not even <think>.\n\n"
        "Email thread:\n"
        + text
    )
    response = client.chat.completions.create(
        model="deepseek-ai/deepseek-r1",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.2,
        max_tokens=500,
    )
    content = response.choices[0].message.content.strip()
    print("🧠 LLM raw response:\n", content)
    # 🛡️ Extract JSON part from the response
    json_start = content.find("{")
    if json_start == -1:
```

```
        raise Exception("❌ LLM did not return any JSON-like content")
    try:
        json_part = content[json_start:]
        result = json.loads(json_part)
    except json.JSONDecodeError as e:
        raise Exception(f"❌ LLM output is not valid JSON: {e}")
    # ✅ Check required fields
    required_fields = ["software", "requester_name", "email", "approved", "approver"]
    for field in required_fields:
        if field not in result:
            raise Exception(f"❌ Missing required field: {field}")

    # 🔄 Add device_id
    result["device_id"] = get_device_id_from_email(result["email"])
    return result
def load_yaml_mapping(filename: str) -> dict:
    with open(filename, "r") as f:
        return yaml.safe_load(f)
def generate_job_file(fields):
    software = fields["software"]
    installer_map = load_yaml_mapping("software_name.yaml")
    installer_filename = installer_map.get(software, None)
    device_id = fields["device_id"]
    hash_value = get_hash(installer_filename)
    shared_path = f"\\\\172.25.80.10\\Share\\{installer_filename}"
    command_map = load_yaml_mapping("installer_command.yaml")
```

```
    install_command = command_map.get(installer_filename, "command_not_found")
    job_data = {
        "software": software,
        "device_id": device_id,
        "hash": hash_value,
        "path": shared_path,
        "install_command": install_command,
        "approved": True
    }
    job_filename = f"job_{device_id}_{software.lower()}.json"
    with open(os.path.join("jobs", job_filename), "w") as f:
        json.dump(job_data, f, indent=2)
    return job_filename
def get_hash(installer_filename):
    with open("hashdb.yaml") as f:
        hashes = yaml.safe_load(f)
    return hashes.get(f"{installer_filename}", "unknown")
def save_report(fields, explanation):
    device = fields.get("device_id", "unknown")
    software = fields.get("software", "unknown")
    report_file = f"reports/report_{device}_{software.lower()}.json"
    with open(report_file, "w") as f:
        json.dump({"steps": explanation}, f, indent=2)
templates = Jinja2Templates(directory="templates")
@app.get("/ticket-info", response_class=HTMLResponse)
async def ticket_info(request: Request):
```

```
    jobs = []
    if os.path.exists("jobs"):
        for fname in os.listdir("jobs"):
            if fname.endswith(".json"):
                with open(os.path.join("jobs", fname)) as f:
                    job = json.load(f)
                    jobs.append({
                        "ticket_id": fname.replace("job_", "").replace(".json", "").upper(),
                        "software": job.get("software"),
                        "device_id": job.get("device_id"),
                        "status": "APPROVED" if job.get("approved") else "REJECTED",
                        "result": "Success" if job.get("approved") else "Failed",
                        "file": fname,
                        "timestamp": datetime.fromtimestamp(os.path.getmtime(os.path.join("jobs",
fname))).strftime("%d-%m-%Y %H:%M:%S")
                    })
    return templates.TemplateResponse("ticket_info.html", {"request": request, "jobs": jobs})
@app.get("/logs", response_class=HTMLResponse)
async def logs_view(request: Request):
    logs = []
    log_dir = "received_logs"
    if os.path.exists(log_dir):
        for fname in os.listdir(log_dir):
            path = os.path.join(log_dir, fname)
            if fname.endswith(".log") and os.path.isfile(path):
                with open(path, "r", encoding="utf-8") as f:
```

```
            content = f.read()
            # Extract ticket ID and status
            ticket_match = re.search(r"Ticket ID:\s*(.*)", content)
            status_match = re.search(r"Execution Status:\s*(.*)", content)
            device_id = ticket_match.group(1).strip() if ticket_match else "unknown"
            status_text = status_match.group(1).strip().lower() if status_match else ""
            if re.search(r"\b(succeed|success|succeeded)\b", status_text):
                status = "Success"
            else:
                status = "Failed"
            logs.append({
                "filename": fname,
                "device_id": device_id,
                "status": status,
            })
    return templates.TemplateResponse("logs.html", {"request": request, "logs": logs})
@app.get("/view-log/{filename}", response_class=PlainTextResponse)
async def view_log(filename: str):
    path = os.path.join("received_logs", filename)
    if not os.path.exists(path):
        return PlainTextResponse("File not found", status_code=404)
    with open(path, "r", encoding="utf-8") as f:
        return PlainTextResponse(f.read())
@app.get("/view-job/{filename}", response_class=PlainTextResponse)
async def view_job(filename: str):
    path = os.path.join("jobs", filename)
```

```
    if not os.path.exists(path):
        return PlainTextResponse("File not found", status_code=404)
    with open(path) as f:
        return PlainTextResponse(f.read())
if __name__ == "__main__":
    import uvicorn
    uvicorn.run("admin_server:app", host="0.0.0.0", port=8080, reload=True)
```

Execution Command:

```
python admin_server
```

3.3.2 index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Admin Dashboard</title>
  <style>
    body {
      font-family: 'Segoe UI', sans-serif;
      background: #e8f5e9;
      margin: 0;
      padding: 0;
    }
    .container {
      display: flex;
      height: 100vh;
    }
    .sidebar {
      width: 200px;
      background: #a5d6a7;
      padding: 20px;
      display: flex;
      flex-direction: column;
      gap: 10px;
```

```
}
.menu-item {
  background: white;
  border-radius: 25px;
  padding: 10px;
  text-align: center;
  font-weight: bold;
  cursor: pointer;
  text-decoration: none;
  color: black;
}
.main {
  flex: 1;
  padding: 20px;
  overflow-y: auto;
}
.header {
  margin-bottom: 20px;
}
.upload-section {
  margin-bottom: 30px;
}
.upload-box {
  background: #c8e6c9;
  border-radius: 10px;
  padding: 20px;
```

```
  text-align: center;
  border: 2px solid #4caf50;
  max-width: 400px;
  margin-bottom: 20px;
}
.upload-box input[type="file"] {
  display: block;
  margin: 10px auto;
}
.upload-btn {
  background: #2e7d32;
  color: white;
  border: none;
  padding: 10px 20px;
  border-radius: 20px;
  cursor: pointer;
}
.live-update {
  background: #c8e6c9;
  border-radius: 10px;
  padding: 20px;
  max-width: 700px;
  margin-bottom: 20px;
}
.step-row {
  display: flex;
```

```
  justify-content: space-between;
  align-items: center;
  margin-bottom: 10px;
}
.step-status {
  display: flex;
  align-items: center;
  gap: 10px;
}
.tick {
  color: green;
  font-size: 18px;
}
.cross {
  color: red;
  font-size: 18px;
}
textarea.output-box {
  width: 100%;
  height: 300px;
  border: 2px solid #388e3c;
  background: #f1f8e9;
  border-radius: 10px;
  padding: 10px;
  font-family: monospace;
  white-space: pre-wrap;
```

```
    resize: none;
   }
  </style>
</head>
<body>
<div class="container">
  <div class="sidebar">
    <div class="menu-item" onclick="location.href='/'">DASHBOARD</div>
    <div class="menu-item" onclick="location.href='/ticket-info'">TICKET INFO</div>
    <div class="menu-item" onclick="location.href='/logs'">LOGS</div>
    <div class="menu-item">LOGOUT</div>
  </div>
  <div class="main">
    <div class="header">
      <h2>Welcome Admin</h2>
    </div>
    <div class="upload-section">
      <div class="upload-box">
        <h3>UPLOAD DOCUMENT</h3>
        <p>Only Upload IS/IT Approval Document</p>
        <input type="file" id="docInput" accept=".docx,.pdf,.txt" />
        <button class="upload-btn" onclick="uploadDoc()">Upload</button>
      </div>
    </div>
    <div class="live-update">
      <h3>LIVE UPDATE</h3>
```

```
<div class="step-row"><span>Document Upload</span>
  <div class="step-status">
    <span class="tick" id="tick-upload" style="display:none;">✔</span>
    <span class="cross" id="cross-upload" style="display:none;">❌</span>
  </div>
</div>
<div class="step-row"><span>Parsing Document</span>
  <div class="step-status">
    <span class="tick" id="tick-parsing" style="display:none;">✔</span>
    <span class="cross" id="cross-parsing" style="display:none;">❌</span>
  </div>
</div>
<div class="step-row"><span>Approval Check</span>
  <div class="step-status">
    <span class="tick" id="tick-approval" style="display:none;">✔</span>
    <span class="cross" id="cross-approval" style="display:none;">❌</span>
  </div>
</div>
<div class="step-row"><span>License & Policy Check</span>
  <div class="step-status">
    <span class="tick" id="tick-license" style="display:none;">✔</span>
    <span class="cross" id="cross-license" style="display:none;">❌</span>
  </div>
</div>
<div class="step-row"><span>Job File Creation</span>
  <div class="step-status">
```

```
        <span class="tick" id="tick-job" style="display:none;">✔</span>
        <span class="cross" id="cross-job" style="display:none;">❌</span>
      </div>
    </div>
  </div>
  <div>
    <h3>AGENTIQ RESPONSE</h3>
    <textarea id="resultOutput" class="output-box" readonly></textarea>
  </div>
 </div>
</div>
<script>
 async function uploadDoc() {
  const file = document.getElementById("docInput").files[0];
  if (!file) return alert("Please select a document");
  const formData = new FormData();
  formData.append("file", file);
  const outputBox = document.getElementById("resultOutput");
  outputBox.value = "Uploading and processing...";



  resetTicksAndCrosses();
  try {
   const res = await fetch(`/upload-email/`, {
    method: "POST",
    body: formData
```

```
    });
    const text = await res.text();
    outputBox.value = text;
    markTick("upload");
    setTimeout(() => markTick("parsing"), 1000);
    setTimeout(() => markTick("approval"), 2000);
    const isBlocked = text.includes("❌ Software") && text.includes("NOT allowed");
    setTimeout(() => {
      if (isBlocked) {
        markCross("license");
        markCross("job");
      } else {
        markTick("license");
        markTick("job");
      }
    }, 3000);
  } catch (err) {
    outputBox.value = "Upload failed. Check console.";
    console.error(err);
  }
}
function markTick(step) {
  document.getElementById(`tick-${step}`).style.display = "inline";
  document.getElementById(`cross-${step}`).style.display = "none";
}
function markCross(step) {
```

```
    document.getElementById(`tick-${step}`).style.display = "none";
    document.getElementById(`cross-${step}`).style.display = "inline";
  }
  function resetTicksAndCrosses() {
    const steps = ["upload", "parsing", "approval", "license", "job"];
    for (let step of steps) {
      document.getElementById(`tick-${step}`).style.display = "none";
      document.getElementById(`cross-${step}`).style.display = "none";
    }
  }
</script>
</body>
</html>
```

3.3.3 logs.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Logs</title>
  <style>
    body { font-family: Arial, sans-serif; padding: 20px; }
    table { width: 100%; border-collapse: collapse; margin-top: 20px; }
    th, td { border: 1px solid #aaa; padding: 10px; text-align: center; }
    th { background-color: #c8e6c9; }
    .success { color: green; font-weight: bold; }
    .failed { color: red; font-weight: bold; }
    button { padding: 5px 10px; cursor: pointer; }
  </style>
</head>
<body>
  <h2>Execution Logs</h2>
  <table>
    <thead>
      <tr>
        <th>File Name</th>
        <th>Device ID</th>
        <th>Status</th>
        <th>Report</th>
```

```
    </tr>
  </thead>
  <tbody>
    {% for log in logs %}
      <tr>
        <td>{{ log.filename }}</td>
        <td>{{ log.device_id }}</td>
        <td>
          {% if log.status == "Success" %}
            <span class="success">Success</span>
          {% else %}
            <span class="failed">Failed</span>
          {% endif %}
        </td>
        <td><a href="/view-log/{{ log.filename }}" target="_blank">👁️</a></td>
      </tr>
    {% endfor %}
  </tbody>
 </table>
</body>
</html>
```

3.3.4 Ticket_info.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Ticket Info</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f1f8e9;
      margin: 0;
      padding: 20px;
    }
    h1 {
      color: #2e7d32;
    }
    table {
      width: 100%;
      border-collapse: collapse;
      margin-top: 20px;
    }
    th, td {
      border: 1px solid #a5d6a7;
      padding: 10px;
      text-align: center;
    }
```

```
    th {
      background: #a5d6a7;
    }
    .success {
      color: green;
      font-weight: bold;
    }
    .failed {
      color: red;
      font-weight: bold;
    }
    .view-button {
      background: none;
      border: none;
      cursor: pointer;
      font-size: 18px;
    }
  </style>
</head>
<body>
<h1>Ticket Info</h1>
<table>
  <tr>
    <th>TICKET NAME</th>
    <th>SOFTWARE</th>
    <th>DEVICE ID</th>
```

```
  <th>TIME STAMP</th>
  <th>REPORT</th>
  <th>RESULT</th>
 </tr>
 {% for job in jobs %}
 <tr>
  <td>{{ job.ticket_id }}</td>
  <td>{{ job.software }}</td>
  <td>{{ job.device_id }}</td>
  <td>{{ job.timestamp }}</td>
  <td>
   <form action="/view-job/{{ job.file }}" method="get" target="_blank">
    <button class="view-button" title="View JSON">👁</button>
   </form>
  </td>
  <td class="{{ 'success' if job.result == 'Success' else 'failed' }}">{{ job.result }}</td>
 </tr>
 {% endfor %}
</table>
</body>
</html>
```

3.4 Client Side Code

Client side code remains the same for both methods. Make sure to follow this folder structure.

```
ClientAgent/
├── bin/                    # Auto-generated build output
├── obj/                    # Auto-generated build cache (ignore)
├── Installers/             # All installer files (.exe/.msi) should go here
│   ├── AdobeReaderSetup.exe
│   └── ChromeInstaller.msi
├── Logs/                   # Store logs for each job
│   ├── log_20250724_1432.txt
│   └── log_20250724_1501.txt
├── config.json             # Must be here — contains admin_server, email, etc.
├── Program.cs              # Main polling, installer, logger code
├── PopupForm.cs            # Form that shows Proceed/Reschedule
├── ClientAgent.csproj      # Project file
```

Note:

1.Create dotnet project:

dotnet new console -n

--> this creates clientagent.csproj and program.cs file , copy the contents and add other files

2. Run the files:

dotnet restore

dotnet build

dotnet build -c Release -o .

dotnet run

3. To make the client side agent as a background process,

1. Build EXE:

Publish the C# project from Visual Studio to a folder (e.g., C:\ClientApp).

Locate the generated .exe (e.g., ClientApp.exe).

2. Test EXE:

Run ClientApp.exe via Command Prompt to ensure it works.

3. Create Scheduled Task:

Open Task Scheduler → Create Basic Task.

Name it (e.g., Run Client App).

4. Set Trigger:

Choose frequency (e.g., daily, every X minutes).

Configure timing.

5. Set Action:

Action: Start a program

Program: C:\ClientApp\ClientApp.exe

6. Configure Properties:

General → Check: "Run whether user is logged on or not"

Triggers → Edit → Set repeat interval if needed

Settings → Enable "Run on demand", "Restart if failed"

7. Save Task:

Enter Windows password when prompted to run in background.

3.4.1 Program.cs

```
using System;
using System.IO;
using System.Net.Http;
using System.Security.Cryptography;
using System.Text.Json;
using System.Threading.Tasks;
using System.Diagnostics;
using System.Windows.Forms;
using Microsoft.Win32;

class Config
{
    public string device_id { get; set; }
    public string admin_server { get; set; }
    public int poll_interval { get; set; }
    public string client_email { get; set; }
    public string admin_name { get; set; }
}

class Job
{
    public string device_id { get; set; }
    public string software { get; set; }
    public string path { get; set; }
```

```
    public string hash { get; set; }
    public string install_command { get; set; }
}

static class Program
{
    static HttpClient http = new HttpClient();
    static Config config;

    [STAThread]
    static async Task Main()
    {
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);

        config = JsonSerializer.Deserialize<Config>(File.ReadAllText("config.json"));
        Directory.CreateDirectory("Logs");
        Directory.CreateDirectory("Installers");

        while (true)
        {
            try
            {
                string url = $"{config.admin_server}/get_job?device_id={config.device_id}";
                var response = await http.GetAsync(url);
```

```
            if (response.IsSuccessStatusCode && response.Content.Headers.ContentLength > 0)
            {
                var jobJson = await response.Content.ReadAsStringAsync();
                var job = JsonSerializer.Deserialize<Job>(jobJson);
                Console.WriteLine($"[+] Job received for {job.software}");

                var popup = new PopupForm(job.software, job.device_id);
                Application.Run(popup);

                if (!popup.UserClickedProceed)
                {
                    string msg = popup.RescheduleDateTime != null
                        ? $"User rescheduled to {popup.RescheduleDateTime}"
                        : "User chose to reschedule.";
                    await SendSimpleLog(job.software, msg);
                    continue;
                }

                string localInstallerName = Path.GetFileName(job.path);
                string localInstallerPath = Path.Combine("Installers", localInstallerName);

                try
                {
                    Console.WriteLine($"[📥] Copying installer from {job.path} to
{localInstallerPath}...");
                    File.Copy(job.path, localInstallerPath, true);
```

```
                Console.WriteLine("[✅] Copy successful.");
            }
            catch (Exception ex)
            {
                Console.WriteLine($"[❌] Failed to copy installer: {ex.Message}");
                await SendDetailedLog(job, "Failed", $"Failed to copy installer from network path: {ex.Message}", false, DateTime.Now, DateTime.Now, "N/A");
                PopupForm.ShowFinalMessage("Copy Failed", $"Could not copy installer:\n{ex.Message}", false);
                continue;
            }

            DateTime jobStartTime = DateTime.Now;

            if (VerifyHash(localInstallerPath, job.hash, out string actualHash))
            {
                if (IsSoftwareAlreadyInstalled(job.software))
                {
                    string skipMessage = $"Skipped installation: {job.software} is already installed.";
                    Console.WriteLine($"[ℹ] {skipMessage}");
                    await SendSimpleLog(job.software, skipMessage);
                    continue;
                }

                string result = InstallSilently(job.install_command, localInstallerPath);
```

```
                DateTime completedAt = DateTime.Now;
                bool isSuccess = result.Contains("ExitCode: 0");

                await SendDetailedLog(job, isSuccess ? "Success" : "Failed", result, true, jobStartTime, completedAt, actualHash);

                PopupForm.ShowFinalMessage(
                    isSuccess ? "Installation Success" : "Installation Failed",
                    isSuccess ? $"{job.software} installed successfully." : $"Installation failed:\n{result}",
                    isSuccess
                );
            }
            else
            {
                DateTime failTime = DateTime.Now;
                await SendDetailedLog(job, "Failed", "Installer hash mismatch. Aborting installation.", false, jobStartTime, failTime, actualHash);
                PopupForm.ShowFinalMessage("Hash Mismatch", "Installer hash mismatch. Aborting installation.", false);
            }
        }
        else
        {
            Console.WriteLine($"[*] No job available. [{DateTime.Now:dd-MM-yyyy HH:mm:ss}]");
```

```
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine($"[!] Error: {ex.Message}");
        }

        await Task.Delay(config.poll_interval * 1000);
    }
}

static bool VerifyHash(string filePath, string expectedHash, out string actualHash)
{
    using var sha256 = SHA256.Create();
    using var stream = File.OpenRead(filePath);
    actualHash = BitConverter.ToString(sha256.ComputeHash(stream)).Replace("-",
"").ToLower();
    Console.WriteLine($"[🔐] Expected: {expectedHash}\n[🔐] Actual:   {actualHash}");
    return actualHash == expectedHash.ToLower();
}

static string InstallSilently(string commandTemplate, string installerPath)
{
    try
    {
        string extension = Path.GetExtension(installerPath).ToLower();
```

```
        string defaultCommand;

        if (extension == ".msi")
        {
            string logFile = Path.Combine("C:\\install_logs",
Path.GetFileNameWithoutExtension(installerPath) + ".log");
            Directory.CreateDirectory("C:\\install_logs");
            defaultCommand = $"msiexec /i \"{installerPath}\" /quiet /norestart /log
\"{logFile}\"";
        }
        else if (extension == ".exe")
        {
            defaultCommand = $"\"{installerPath}\" /quiet /norestart";
        }
        else
        {
            return $"Unknown installer type: {extension}. Only .exe and .msi are supported.";
        }

        string command = string.IsNullOrWhiteSpace(commandTemplate)
            ? defaultCommand
            : commandTemplate.Replace("{installer}", installerPath);

        Console.WriteLine($"[⚙] Final Install Command: {command}");

        var process = Process.Start(new ProcessStartInfo
```

```
        {
            FileName = "cmd.exe",
            Arguments = $"/C {command}",
            RedirectStandardOutput = true,
            RedirectStandardError = true,
            UseShellExecute = false,
            Verb = "runas"
        });

        process.WaitForExit(300000); // Wait 5 minutes max
        string output = process.StandardOutput.ReadToEnd() +
process.StandardError.ReadToEnd();
        return $"ExitCode: {process.ExitCode} -
{ExplainExitCode(process.ExitCode)}\n{output}";
    }
    catch (Exception ex)
    {
        return $"Install failed: {ex.Message}";
    }
}

static string ExplainExitCode(int code)
{
    return code switch
    {
        0 => "Installation succeeded.",
```

```
        1603 => "Fatal error: Already installed, permission issue, invalid command, or blocked by antivirus.",
        _ => $"Unknown exit code: {code}"
    };
}

static bool IsSoftwareAlreadyInstalled(string softwareName)
{
    string uninstallKey = @"SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall";
    using var baseKey = Registry.LocalMachine.OpenSubKey(uninstallKey);
    if (baseKey == null) return false;

    foreach (string subkeyName in baseKey.GetSubKeyNames())
    {
        using var subkey = baseKey.OpenSubKey(subkeyName);
        string displayName = subkey?.GetValue("DisplayName")?.ToString();
        if (!string.IsNullOrWhiteSpace(displayName) &&
            displayName.Contains(softwareName, StringComparison.OrdinalIgnoreCase))
        {
            return true;
        }
    }
    return false;
}

static async Task SendSimpleLog(string software, string message)
```

```
    {
        string logPath = $"Logs/{software}_{DateTime.Now:yyyyMMdd_HHmmss}.txt";
        await File.WriteAllTextAsync(logPath, message);

        var content = new MultipartFormDataContent();
        content.Add(new ByteArrayContent(await File.ReadAllBytesAsync(logPath)), "logfile",
Path.GetFileName(logPath));

        var res = await http.PostAsync($"{config.admin_server}/send_log", content);
        Console.WriteLine($"[📤] Log sent. Status: {res.StatusCode}");
    }

    static async Task SendDetailedLog(Job job, string status, string installOutput, bool
hashVerified, DateTime jobSentAt, DateTime completedAt, string actualHash)
    {
        string hashStatus = hashVerified ? "✔ Verified" : $"Mismatch (Expected: {job.hash},
Got: {actualHash})";
        string fileName = Path.GetFileName(job.path);
        string ticketId = job.device_id;
        string logFileName = $"{job.software.ToLower()}_{(status == "Success" ? "install" :
"error")}_{ticketId}.log";
        string logPath = $"Logs/{logFileName}";

        string logText = $@"
Ticket ID: {ticketId}
Software: {job.software}
```

```
Client Mail ID: {config.client_email}
Admin Name: {config.admin_name}
Job Sent At: {jobSentAt:dd-MM-yyyy HH:mm:ss}
{(status == "Success" ? "Installation Completed At" : "Failure Detected At")}:
{completedAt:dd-MM-yyyy HH:mm:ss}
Hash Verification: {hashStatus}
Installer File: {fileName}
Execution Status: {(status == "Success" ? "Install succeeded" : "Install failed")}
{(status == "Success" ? "" : $"Error: {installOutput.Trim()}")}
Log File: {logFileName}
".Trim();

        await File.WriteAllTextAsync(logPath, logText);

        var content = new MultipartFormDataContent();
        content.Add(new ByteArrayContent(await File.ReadAllBytesAsync(logPath)), "logfile",
logFileName);

        var res = await http.PostAsync($"{config.admin_server}/send_log", content);
        Console.WriteLine($"[📤] Detailed log sent. Status: {res.StatusCode}");
    }
}
```

3.4.2 popupform.cs

```
using System;
using System.Drawing;
using System.Windows.Forms;

public class PopupForm : Form
{
    public bool UserClickedProceed { get; private set; } = false;
    public DateTime? RescheduleDateTime { get; private set; } = null;

    private Label ticketLabel;
    private Label messageLabel;
    private DateTimePicker dateTimePicker;
    private Button confirmButton;

    public PopupForm(string softwareName, string ticketId)
    {
        this.Text = "Installation Notification";
        this.Size = new Size(520, 320);
        this.StartPosition = FormStartPosition.CenterScreen;
        this.FormBorderStyle = FormBorderStyle.FixedDialog;
        this.MaximizeBox = false;
        this.MinimizeBox = false;
        this.BackColor = Color.FromArgb(213, 243, 201); // light green
```

```
var headerPanel = new Panel
{
    BackColor = Color.FromArgb(185, 229, 160),
    Height = 40,
    Dock = DockStyle.Top
};

var headerLabel = new Label
{
    Text = "Installation Notification",
    Font = new Font("Segoe UI", 11, FontStyle.Bold),
    AutoSize = true,
    Location = new Point(10, 10)
};
headerPanel.Controls.Add(headerLabel);
Controls.Add(headerPanel);

ticketLabel = new Label
{
    Text = $"TICKET ID: {ticketId}",
    Font = new Font("Segoe UI", 10, FontStyle.Bold),
    Location = new Point(20, 60),
    AutoSize = true
};
Controls.Add(ticketLabel);
```

```
        messageLabel = new Label
        {
            Text = $"Dear user, as per request, {softwareName} installation is scheduled.\nPlease choose one of the options below.",
            Font = new Font("Segoe UI", 10),
            Location = new Point(20, 100),
            Size = new Size(460, 40)
        };
        Controls.Add(messageLabel);

        // Proceed Button
        var proceedButton = new Button
        {
            Text = "Proceed",
            Location = new Point(100, 160),
            Size = new Size(120, 40),
            BackColor = Color.FromArgb(250, 250, 180),
            FlatStyle = FlatStyle.Flat
        };
        proceedButton.Click += (s, e) =>
        {
            UserClickedProceed = true;
            this.Close();
        };
        Controls.Add(proceedButton);
```

```
// Reschedule Button
var rescheduleButton = new Button
{
    Text = "Reschedule",
    Location = new Point(260, 160),
    Size = new Size(120, 40),
    BackColor = Color.FromArgb(250, 250, 180),
    FlatStyle = FlatStyle.Flat
};
rescheduleButton.Click += (s, e) =>
{
    ToggleDateTimePicker(true);
};
Controls.Add(rescheduleButton);

// Hidden datetime picker
dateTimePicker = new DateTimePicker
{
    Format = DateTimePickerFormat.Custom,
    CustomFormat = "dd-MM-yyyy hh:mm tt",
    Location = new Point(100, 220),
    Width = 300,
    Visible = false
};
Controls.Add(dateTimePicker);
```

```
        // Confirm Button (for reschedule)
        confirmButton = new Button
        {
            Text = "Confirm Reschedule",
            Location = new Point(170, 260),
            Size = new Size(150, 35),
            BackColor = Color.LightGray,
            Visible = false
        };
        confirmButton.Click += (s, e) =>
        {
            RescheduleDateTime = dateTimePicker.Value;
            MessageBox.Show($"Rescheduled to: {RescheduleDateTime}", "Rescheduled",
MessageBoxButtons.OK, MessageBoxIcon.Information);
            this.Close();
        };
        Controls.Add(confirmButton);
    }

    private void ToggleDateTimePicker(bool show)
    {
        dateTimePicker.Visible = show;
        confirmButton.Visible = show;
    }

    public static void ShowFinalMessage(string title, string message, bool isSuccess)
```

```
  {
     MessageBox.Show(message, title,
        MessageBoxButtons.OK,
        isSuccess ? MessageBoxIcon.Information : MessageBoxIcon.Error);
  }
}
#popupform.cs file for displaying popup
```

3.4.3 config.json

```
{
 "device_id": "device123",
 "admin_server": "http://172.25.80.48:8080",
 "poll_interval": 30,
 "client_email": "123@gmail.com",
 "admin_name": "Admin1"
}
```

3.4.4 C# project source file content

```
<Project Sdk="Microsoft.NET.Sdk.WindowsDesktop">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net9.0-windows</TargetFramework>
    <UseWindowsForms>true</UseWindowsForms>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

Execution Command

```
cd C:\ClientAgent
dotnet build
dotnet run
```

4. Future Enhancements

1. **Mail synchronization :** Synchronize with mails directly to fasten the entire process.
2. **Metrics :** Display metrics on dashboard.
3. **Reschedule installation :** Allow clients to reschedule the installation to a desired time.

5. Credentials

Janani S:

- VM IP: 172.25.80.46
- Username: LntEle30
- Password: nt!2530()

Harishmaa M:

- VM IP: 172.25.80.48
- Username: LntEle32
- Password: nt!2532()

Jayashree K A:

- VM IP: 172.25.80.47
- Username: LntEle31
- Password: nt!2531()

New VM:

- VM IP: 172.25.80.52
- Username: LntEle35
- Password: nt!2535()

6. References

1. NVIDIA NeMo Agent Toolkit Documentation

   https://docs.nvidia.com/nemo/agent-toolkit

2. AgentIQ GitHub Repository (NVIDIA NeMo Agents)

   https://github.com/NVIDIA/NeMo-Agents

3. FastAPI Official Documentation

   https://fastapi.tiangolo.com

4. Python Standard Library Docs

   https://docs.python.org/3.11/

5. Pandas Documentation (for Excel processing)

   https://pandas.pydata.org/docs/